

# Trabalho Prático da Unidade 1

## 1 Contexto

Quando dividimos um problema entre várias unidades de execução, é tentador imaginar que usar o dobro de unidades corta o tempo pela metade. Isso raramente acontece, e há pelo menos quatro razões distintas para isso:

1. Criar uma unidade de execução custa tempo, e esse custo não é o mesmo para processos e para threads.
2. O tempo total do conjunto não é a média das unidades: é o tempo do **mais lento**. Se a divisão do trabalho for desigual, os demais ficam ociosos esperando.
3. A partir de certo ponto, não há mais CPUs livres, e o sistema operacional passa a **revezar** as unidades de execução em vez de executá-las simultaneamente.
4. Sempre existe uma parte do programa que não é dividida (ler parâmetros, criar unidades de execução, juntar resultados).

Neste trabalho você deve construir uma aplicação para **medir** cada um desses quatro efeitos separadamente, usando um problema cuja carga de trabalho é propositalmente **irregular**.

## 2 Descrição matemática do problema

Dado um conjunto  $N$  de todos números inteiros positivos no intervalo  $[A, B]$ , sendo  $A, B \in \mathbb{N}^*$ , com  $A < B$ , percorrer todos os elementos deste intervalo e calcular, **para cada um deles**, o seguinte problema computacional  $s(\cdot)$ :

$$n_{k+1} = \begin{cases} \text{Finaliza,} & \text{se } n_k = 1 \\ \frac{n_k}{2}, & \text{se } n_k \text{ for par} \\ 3n_k + 1, & \text{se } n_k \text{ for ímpar} \end{cases}$$

onde  $n_k$  é o número inteiro  $n$  na iteração  $k$ , com  $n_0$  como o número inicial fornecido. O retorno da função  $s(\cdot)$  é a quantidade de passos executados até o algoritmo finalizar sua execução.

**1- Exemplo prático da função  $s(\cdot)$ :**  $s(6) = 8$ , pois a sequência exige exatamente 8 passos para alcançar o número 1:

Passo 1: 6 é par.  $6/2$ . Resultado 3  
 Passo 2: 3 é ímpar.  $3 * 3 + 1$ . Resultado 10  
 Passo 3: 10 é par.  $10/2$ . Resultado 5  
 Passo 4: 5 é ímpar.  $3 * 5 + 1$ . Resultado 16  
 Passo 5: 16 é par.  $16/2$ . Resultado 8  
 Passo 6: 8 é par.  $8/2$ . Resultado 4  
 Passo 7: 4 é par.  $4/2$ . Resultado 2  
 Passo 8: 2 é par.  $2/2$ . Resultado 1

**2- Exemplo prático do problema completo:** Se  $A = 10$  e  $B = 20$ , é necessário calcular  $s(10), s(11), s(12), \dots, s(20)$ .

## 3 Descrição do problema computacional

Você deve escrever **um único programa** chamado **varredor** que resolve o problema matemático descrito anteriormente. O programa pode ser executado inteiramente de forma sequencial ou se dividir ou entre múltiplas threads filhas ou múltiplos processos filhos. Nestes casos, somente os filhos realizam

os cálculos de  $s(\cdot)$  sobre os números do intervalo  $[A, B]$ . Os filhos devem dividir a computação dos números entre si conforme especificado pelo usuário.

A linha de execução do programa **varredor** é:

```
./varredor <A> <B> <W> <modo> <particao> <arquivo_saida>
```

onde

| Argumento     | Valores            | Significado                                                                                                   |
|---------------|--------------------|---------------------------------------------------------------------------------------------------------------|
| W             | inteiro $\geq 1$   | Número de threads ou processos filhos que devem realizar a computação. Se $W=1$ , então é o código sequencial |
| modo          | thread ou processo | Se irá criar threads ou processos filhos, caso $W>1$                                                          |
| particao      | bloco ou ciclico   | Como dividir e calcular o intervalo de números, detalhado abaixo                                              |
| arquivo_saida | caminho            | Onde gravar a linha de resultado                                                                              |

Para entender a partição, considere cada thread ou processo filho como  $w$ , com  $w = \{0, 1, \dots, W\}$ . Assim:

- **bloco** — Cada thread ou processo filho  $w$  realiza a computação uma faixa contígua de aproximadamente  $L/W$  inteiros consecutivos, sendo  $L$  a quantidade de números a serem computados com a função  $s(\cdot)$ . Você decide como tratar o resto da divisão quando  $L$  não é múltiplo de  $W$ , e deve **justificar sua escolha** no relatório.
- **ciclico** — Cada thread ou processo filho  $w$  computam os números tal que  $w = \text{NÚMERO} \bmod W$ . Ou seja, as unidades de execução se intercalam elemento a elemento.

Ambas as estratégias distribuem exatamente a mesma quantidade de inteiros. A diferença está inteiramente em *quais* inteiros cada thread ou processo filho recebe e processa.

**1- Exemplo prático:** Considere a seguinte execução:

```
./varredor 12 20 3 thread bloco parcial.csv
```

Neste caso, **varredor** criará 3 threads filhos que realizam o processamento de todos os números de 12 ao 20 (9 números no total). Cada thread receberá  $9/3$  elementos consecutivos. A thread  $w=0$  realizará  $s(12)$ ,  $s(13)$  e  $s(14)$ . A thread  $w=1$  realizará  $s(15)$ ,  $s(16)$  e  $s(17)$ . A thread  $w=2$  realizará  $s(18)$ ,  $s(19)$  e  $s(20)$ . O resultado final, detalhado posteriormente, será salvo no arquivo parcial.csv

**2- Exemplo prático:** Considere a seguinte execução:

```
./varredor 12 20 1 processo ciclico parcial.csv
```

Neste caso, **varredor** não criará filhos porque  $W=1$ . Por este mesmo motivo, ele pode computar sequencialmente todos os números e ignorar **ciclico**. O resultado final, detalhado posteriormente, será salvo no arquivo parcial.csv

## 4 Saída do Programa

O programa pai deve combinar resultados dos filhos, se existirem, e gravar **uma única linha** (CSV ou TXT) em <arquivo\_saida> com exatamente este formato:

```
modo, particao, W, L, tempo_total, tempo_max_filho, tempo_min_filho,
tempo_agregacao
```

tal que

| variável        | definição                                                                      |
|-----------------|--------------------------------------------------------------------------------|
| modo            | thread ou processo, conforme descrito anteriormente                            |
| particao        | bloco ou ciclico, conforme descrito anteriormente                              |
| W               | numero de threads ou processos, conforme descrito anteriormente                |
| L               | quantidade de números analisados (B-A+1)                                       |
| tempo_total     | Tempo de execução do pai, do início ao fim                                     |
| tempo_max_filho | Tempo do filho com menor tempo de processamento                                |
| tempo_min_filho | Tempo do filho com menor tempo de processamento                                |
| tempo_agregação | Tempo do pai após a finalização da execução de todos os filhos e o seu término |

No caso sequencial ( $W=1$ ), as informações relacionadas aos filhos e o tempo de agregação podem preenchidas com números negativos (como -1) para representar a ausência de filhos.

## 5 Parâmetros de Entrada

Os testes serão executados em diversas configurações. Alguns parâmetros de entrada são constantes em todos os casos de teste, e outros variáveis.

Para os parâmetros constantes, considere **M** o número formado pelos **seis últimos dígitos da sua matrícula**. Por exemplo, para a matrícula 20261002739,  $M=002739$ . Assim, temos:

| Parâmetros Constantes   | Definição             |
|-------------------------|-----------------------|
| A (início do intervalo) | $A = 100.000.000 + M$ |
| B (fim do intervalo)    | $B = 8.000.000.000$   |

Os parâmetros variáveis são W, modo, particao e arquivo\_saida, da seguinte forma

| Parâmetros Variáveis | Definição                |
|----------------------|--------------------------|
| W                    | 1,2,4 ou 8               |
| modo                 | thread ou processo       |
| particao             | bloco ou ciclico         |
| arquivo_saida        | nome do arquivo de saída |

É necessário uma execução em cada configuração possível. Portanto, considerando todos os parâmetros, é necessário realizar um total 16 testes diferentes: Cada um dos 4 **w** devem ser combinados com 2 tipos de **modo** e 2 tipos de **particao** ( $16 = 4*2*2$ ).

## 6 Requisitos obrigatórios de implementação

1. Escrito em **C** ou **C++**, compilando sem erros com `g++ -O3 -Wall -pthread` (ou gcc equivalente). Código compilável através de Makefile funcional.
2. No modo **thread**, use threads POSIX (`pthread_create` / `pthread_join`) ou `std::thread`. No modo **processo**, use `fork()` e `wait()/waitpid()`.
3. Cada **thread** ou **processo** filho deve medir e reportar o seu próprio tempo de execução, contado do início do seu trabalho útil até o fim dele.

4. Para  $W = 1$ , o processo inicial não precisa criar threads ou processos filho. Ele deve ignorar o modo e particao. Basta ele realizar a computação de  $s(.)$  para todos os elementos do vetor de forma sequencial (um após o outro).
5. Para  $W \geq 2$ , o processo ou thread pai não realiza computação de  $s(.)$  para os números no intervalo. Os filhos são os responsáveis pela computação. O pai deve medir o **tempo total**, contado desde imediatamente antes de criar a primeira unidade de execução até depois de coletar todos os resultados. Deve também medir separadamente o **tempo de agregação** (o que ele gasta depois que a última unidade de execução terminou).
6. **Cada thread ou processo filho** computa suas informações individualmente. Não é permitido que duas unidades de execução trabalhador escrevam na mesma posição de um endereço de memória compartilhado (para threads) ou nem no mesmo arquivo (para processos).
  - No modo **thread**: cada thread filha escreve **exclusivamente na posição  $i$**  de vetores globais previamente dimensionados, caso exista vetor compartilhado entre threads. O pai só lê essas posições **depois** de todos os **join**, i.e, após todas as threads filhas terminarem sua execução.
  - No modo **processo**: cada processo filho grava um arquivo próprio **parcial\_<i>.txt** com seus resultados parciais e seu tempo. O pai só lê esses arquivos **depois** de todos os **wait**, i.e, após todos os processos filhos terminarem sua execução.
7. Use variáveis de 64 bits (como unsigned long long em C/C++) para representar os números inteiros. Poderá ocorrer overflow caso utilize inteiros de 32 bits.
8. O pai combina os resultados parciais de seus filhos e grava **uma única linha** (CSV ou TXT) em <arquivo\_saida> com exatamente este formato:  
  

```
modo, particao,W,L,tempo_total,tempo_max_filho,tempo_min_filho,
tempo_agregacao
```
9. O tempos devem ser registrado em segundos com **notação científica com precisão de duas casas**.

## 7 Relatório e análise

O relatório deve conter, no mínimo:

- **Identificação:** Nome completo e matrícula.
- **Ambiente de teste** conforme descrito acima.
- **Respostas às questões da seção 7.1:** cada resposta deve citar **números concretos obtidos por você** e relacioná-los a conceitos de Sistemas Operacionais e demais disciplinas do curso. Respostas genéricas, sem referência aos seus dados, não pontuam.

### 7.1 Questionamentos

#### Questão 1 — Desbalanceamento

Compare **bloco** e **ciclico**, **thread** e **processos**, para um mesmo  $w=[1, 2, 4, 8]$ , mostre os resultados e faça seus comentários. Explique, a partir da natureza da função  $s(n)$ , se as duas estratégias (particao e modo) produzem tempos de execução muito distintos. O tempo total do conjunto se aproxima do tempo do filho mais lento ou do mais rápido? Justifique as respostas.

#### Questão 2 — Onde o desempenho é interrompido

Discuta pra qual(s) valor(s) de  $W$  a redução do tempo passa a não ser significativo e indique as razões disso acontecer. Compare-o com o número  $C$  de CPUs físicas e lógicas (se houver) da sua máquina. Em particular, descreva o que acontece com o tempo quando  $W = 8$ : ele piora, melhora ou fica estável em relação às demais configurações de  $W$ ? Justifique.

### Questão 3 — Granularidade e a parte não paralelizável

Estime a fração do tempo que **não** é reduzida ao aumentar  $W$ . Depois, indique concretamente quais trechos do **seu** código compõem essa fração. Justifique.

### Questão 4 — Decisões de projeto

Justifique como você tratou o resto da divisão de  $L$  por  $W$  na partição **bloco**, e qual o impacto dessa escolha na razão de desbalanceamento de carga para  $W$  grande. Existe uma solução que possa melhorar este a sua estratégia? Como seria?

---

## 8 Entrega

Um único arquivo compactado em formato **.zip** contendo:

1. Todo o código-fonte do varredor e o Makefile.
2. O(s) arquivo(s) CSV brutos com **todas** as execuções (não apenas as médias).
3. O relatório em PDF.
4. Um vídeo de até 2 minutos demonstrando a compilação e comentando resultados do seu relatório.  
**O link deve ser público e verificado.**

### 8.1 Receberá nota mínima o aluno que:

- Submeter código da aplicação que não compila;
- Alterar o problema especificado ou não seguir as instruções comentadas neste documento;
- Não cumprir os itens obrigatórios de todos os componentes;
- Utilizar parâmetros individuais que não correspondam à sua matrícula;
- Não disponibilizar link público e acessível para a apresentação;
- Cometer suspeita de plágio;
- Submeter fora do horário estipulado ou por meio que não o SIGAA.

Exceções ou casos omissos devem ser tratados diretamente com o Professor da disciplina.

---

## 9 Checkpoint

O checkpoint consiste na apresentação da solução sequencial desta atividade no dia destinado à entrega desta atividade (ver cronograma). A versão sequencial deve seguir todos os requisitos descritos da Seção 1 a Seção 6. Não é necessário submeter vídeo ou relatório.

No dia do checkpoint, haverá **encontro síncrono virtual**. Individualmente, cada aluno apresentará o seu código compilando e executando para um ou mais casos de teste indicados pelo professor, com notas variando de 0.0 a 10.0. A saída do código deve mostrar o descrito na Seção 4 deste documento, a citar:

```
modo, particao,W,L,tempo_total,tempo_max_filho,tempo_min_filho,
tempo_agregacao
```

A apresentação acontecerá somente no dia indicado. A nota mínima será atribuída ao alunos que não realizarem a apresentação na data indicada.

A não contemplação adequada dos requisitos do checkpoint não elimina a obrigatoriedade de submeter a versão sequencial do programa na entrega final do trabalho. Contudo, a pontuação referente ao checkpoint não será atribuída.